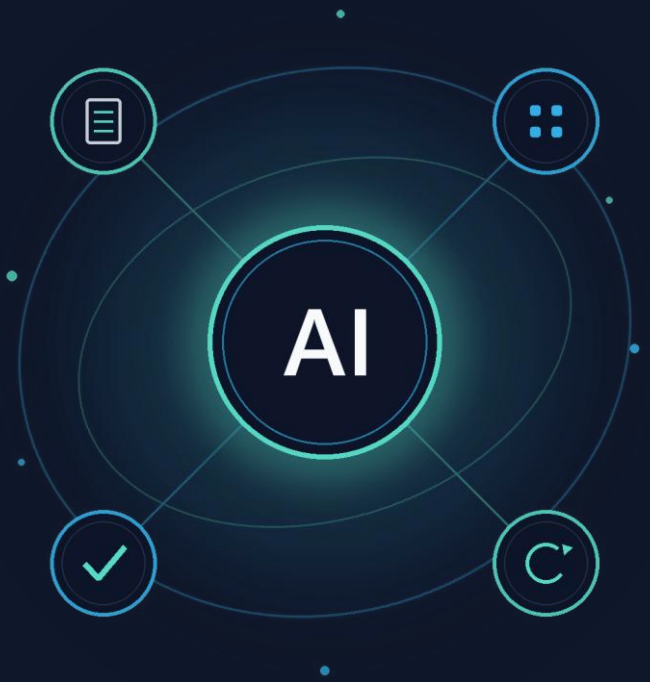
當 AI 加入團隊

打造可審查、可驗證、會自我改善的 AI 開發工作流



James Tsai (蔡景翔)

Microsoft® MVP Most Valuable Professional

Hello World Dev Conference 2026

iThome 鐵人賽
https://ithelp.ithome.com.tw/users/20126714/ironman



James Tsai
https://jamestsai.tw

GitHub
https://github.com/JamestsaiTW

DotBlog
James's Tech & Life Blog
https://dotblogs.com.tw/jamestsai

MVP
Microsoft®
Most Valuable Professional
https://mvp.microsoft.com/zh-TW/MVP/profile/57b5922b-e357-e511-810a-c4346bac0abc

工程團隊問題處理的 SOP 流程

在 AI 還沒開始介入前，一個問題是這樣被處理的

1 Jira
開單
問題與需求
進到一張票

2 Repo
改程式
工程師在本機
分支上修改

3 裝置驗證
驗證
模擬器或真機
最終真機確認

4 GitLab / GitHub
MR / PR
驗證過才送審
同儕審查程式碼

5 CI/CD
自動檢查
建置與測試
擋住明顯錯誤

6 發佈
合併上線
通過後合併
進入發佈流程

每一步都靠人

知識在資深人員腦中 · 驗證靠記憶 · 交接靠口頭

工程團隊手上已經有的東西

AI 還沒來之前，團隊就已經有這些...

系統與工具

Jira
任務與缺陷的入口

GitLab / GitHub
程式碼、版本與 MR / PR 審查

CI/CD
自動建置與測試

測試裝置
手上的真機，驗證行為

模擬器 + 小工具
搭配自動腳本測試

自動化流程
既有的腳本與排程

人與知識

資深人員的經驗
只存在腦中與過往的人與人的對話裡

SOP 與文件
散落在各處、容易過期

Log 與紀錄
出事時才去翻

工具都齊了；缺的是把「人與知識」變成可重複的流程

怎麼把 AI 接進去：沿用同一條路，只新增一步

唯一新增的一步

1 Jira 票

任務從票開始

用自己的帳號
短效 cookie
用完即刪

2 載入指引

AI 先讀規則

放 AGENTS.md
指向規則的 repo
該檔被 git 排除

3 修改程式

先不送 MR

AI 在產品 repo 修改
還在本機分支
規則的改動另走 MR

4 驗證

沿用原本的關卡

模擬器或真機
AI 透過 skill 腳本做
輸出附回票上

5 MR + CI

驗證過才送審

驗證通過才開 MR
由人審查程式碼
CI 跑自動檢查

只多「載入指引」一步，其餘沿用上一頁的流程與關卡

這場你可以帶走這 5 件事...

1. 把個人 Prompt 技巧，變成團隊可維護、可審查的工作流
2. 讓 AI 的「完成」可觀察、可驗證，不只是「看起來完成」
3. 最小上下文、工具權限、敏感資料隔離的安全設計原則
4. 「執行 → 驗證 → 反思 → 審核 → 演進」的 Loop (Human in the Loop)
5. 一份用 Git、文件、腳本就能立刻試行的最小導入藍圖

👋 先舉個手

你的團隊，AI 的 Prompt 現在放在哪裡？

A

每個人的聊天視窗

人~~走了~~就沒了

B

有共用文件，沒人維護

越寫越過期

C

進 Git，經 PR 審查

跟程式碼一樣被管理

👋 請舉手

讓團隊中每個成員都能用同一套方式進行工作

用一個 Git repo 就替 AI 補上四項能力後加入團隊

1 路由

找對規則

入口 manifest 讓 AI 只載入這次任務需要的規則與知識

`aiorchestrations.json`

2 行為契約

完成要有證據

先定完成條件；宣告完成時要附上指令與實際輸出，未驗證的要標示

`docs/operating-contract.md`

3 安全邊界

動作有邊界

Skill 宣告允許的動作與副作用；憑證、log 等執行期資料留在 repo 之外

`skills/` · `scripts/`

4 自我改善閉環

經驗經人審核回流

回顧只產生提案，由人核准後，才升級成共用的 Knowledge 或 Skill

`session-retrospective/`

全部是 Git + Markdown + 腳本，每一次變更都能用 Pull Request 審查

一個問題，為什麼 阿爾法 無法重現 貝爾塔 的成功？

工程師 貝爾塔（週一）

- 一串很厲害的 Prompt
- 三兩下解掉棘手問題
- 收工，關掉聊天視窗

工程師 阿爾法（週二）

- 遇到類似的問題
- 不知道 **貝爾塔** 怎麼問 AI 的
- 不知道參考了什麼資訊
- 不知道怎麼確認修好

不是 AI 不夠聰明，是「經驗沒有被留下來」

工程團隊導入 AI 大致會遇到這三類問題：

1

經驗鎖在個人對話

症狀

Prompt 只活在聊天視窗

後果

不可複製、不可審查、人走就沒了

2

完成由 AI 自己宣告

症狀

說「已完成、已驗證」卻沒跑檢查

後果

「看起來完成」≠「完成」

3

權限與資料沒有邊界

症狀

log、憑證、工具權限散落各處

後果

洩漏與誤操作的風險

三類問題不分先後，都是「**治 理**」問題，不是換更強的模型就能解決

問題不在模型不夠強

而在 AI 的行為缺乏可治理的框架

刻意輕量：用工程團隊已經會的工具

刻意不引入

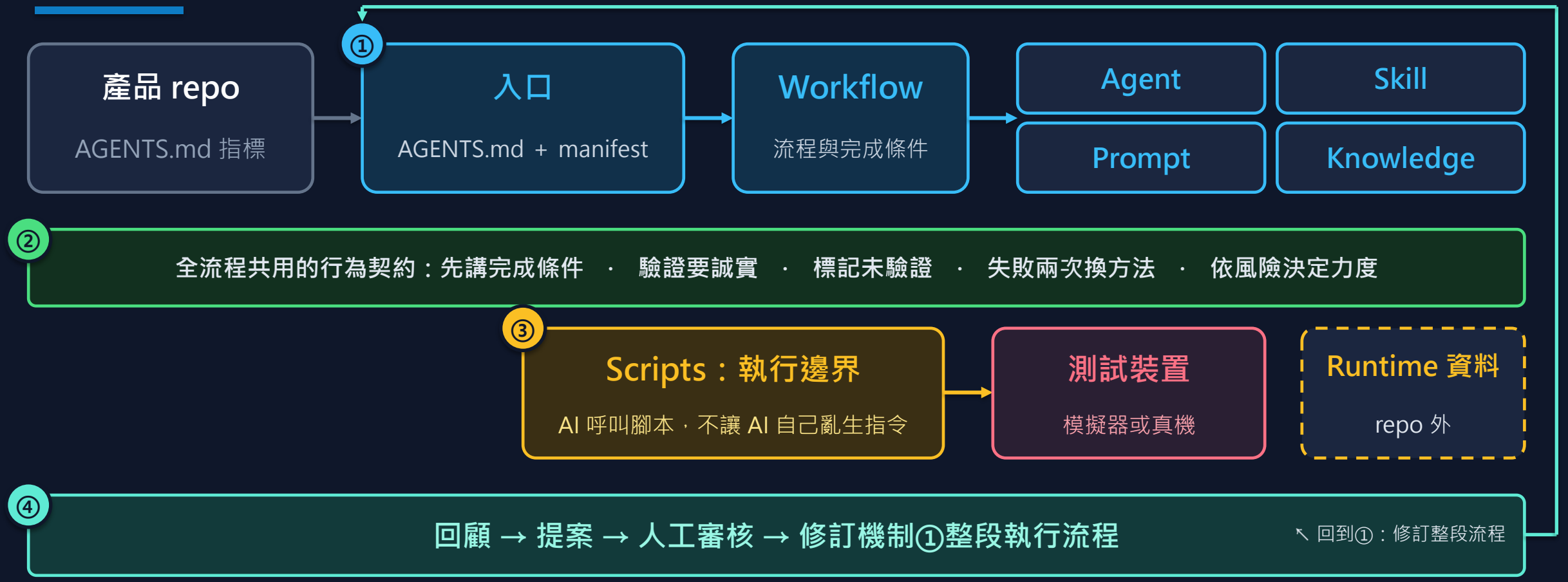
- ✕ 常駐服務
- ✕ 向量資料庫
- ✕ 新的平台或框架

只用三樣

- ✓ Git：版本與審查（PR）
- ✓ Markdown：流程與知識
- ✓ 腳本：可重複的動作

AI 協作 = 一份可以被 Pull Request 審查的工程資產

AIOrchestrations 概觀



①路由（機制 1） ②行為契約（機制 2） ③安全邊界（機制 3） ④自我改善閉環（機制 4）

設計 AIOrchestrations 時的取捨

選擇（得到）		代價（付出）
✓ 每一次變更都能 Pull Request 審查	換來	檢索彈性不如向量資料庫
✓ 零新增基礎設施與維運	換來	路由與文件品質需要人維護
✓ 人與 AI 讀同一份來源	換來	規模變大後要靠規範控制膨脹

先選「可審查」，再視需要加「**工具**」

AIOrchestrations 概觀 – 機制 1



①路由（機制 1） ②行為契約（機制 2） ③安全邊界（機制 3） ④自我改善閉環（機制 4）

分層與路由：每個任務都有可走的歸屬

目錄	職責
workflows/	端到端流程：輸入 → 步驟 → 輸出 → 完成條件
agents/	需要協調多個能力的角色設定
skills/	單一、可重複使用的能力（含安全邊界）
prompts/	固定用途的提示樣板
scripts/	真正會被執行的輔助腳本
knowledge/	已審查、可重複使用的領域知識
docs/ · schemas/	給人與 AI 的說明；驗證檔案格式的 JSON Schema

任何成員，無論是 **人** 還是 **Agent**，不用讀完整個 repo，就能發現該到哪找

全部讀進來，只徒增使用的 Token 數

全部載入

- 一個巨大的說明檔
- 不相關規則互相干擾
- 把 A 平台的判斷套到 B 平台

Router 驅動

- 入口 manifest 先比對任務
- 只載入最小必要上下文
- 有意識地選擇，不是盡量少讀

分層與路由的設計，亦可降低無關規則混入的風險

每次載入有預設額度：先讀少、視需要再加量讀

1	**產品 repo 的指引檔** AGENTS.md 指標，只指路、不含規則	1 份
2	**AIOrchestrations 的 AGENTS.md** 共用入口說明	1 份
3	**主要 workflow 或 skill** 這個任務要用的那一個	1 份
4	**次要知識文件** 需要時才加	0 ~ 2 份
5	**團隊記憶查詢（選用）** 只當參考，不取代審查	最多 5 筆

額度是預設，不是上限

- 任務確實需要更多背景，就大方地多讀，不是一定。
- 重點是有意義，超過要有理由。



採用後 Token 用量顯著降低

題目：發生 ATM 機台 (Windows 平台) 無法正常出鈔問題，AI 透過指示進行問題排除。

A | 採用分層與路由設計

190k 輸入 token（3 次平均）

AGENTS.md 約 4 KB，入口 manifest 路由

- ✓ 3/3 抵達正確結論
- ✓ 3/3 沒載入無關平台的 skill、都標示未驗證

B | 未採用：全量載入對照

300k 輸入 token（3 次平均）

AGENTS.md 約 80 KB，每次請求都帶著

- ✓ 3/3 抵達正確結論
- ✓ 3/3 沒載入無關平台的 skill、都標示未驗證

答案一樣正確；路由載入的輸入 token 少約 37%

註：1 個題目、1 個模型、各跑 3 次樣本（約 144 KB）；不能推論到更大的 repo 或更難的題目

A 是怎麼走到答案的？

同一題「ATM-W 的機台無法出鈔」：入口設計不同，知識進上下文的方式也不同。

A | 採用分層與路由設計載入

1. AGENTS.md 約 4 KB：指向入口 manifest
2. manifest 路由到相關 workflow / skill
3. 按需讀取知識；需要時再讀樣本 log

IsDispenseEnabled=0 | gate suspected
硬體未驗證

B | 未採用：本次全量載入對照

1. 移除入口 manifest
2. 全部 Markdown 合併為單一 AGENTS.md（約 80 KB）
3. 每次請求都帶整份指引，再找本題內容

相同結論：IsDispenseEnabled=0
硬體未驗證

本次兩邊都得到相同結論；差異是「按需載入」與「每次帶入整份指引」

註：B 是本次自建的未採用對照（全量 AGENTS.md），不代表一般作法；僅 1 題、1 模型、各 3 次

每次載入有預設額度：先讀少、視需要再加量讀

1	產品 repo 的指引檔 AGENTS.md 指標，只指路、不含規則	1 份
2	AIOrchestrations 的 AGENTS.md 共用入口說明	1 份
3	主要 workflow 或 skill 這個任務要用的那一個	1 份
4	次要知識文件 需要時才加	0 ~ 2 份
5	團隊記憶查詢（選用） 只當參考，不取代審查	最多 5 筆

額度是預設，不是上限

- 任務確實需要更多背景，就大方地多讀，不是一定。
- 重點是有意義，超過要有理由。

對照 該 案例

那題讀了約 10 個檔，超過額度；理由是要樣本 log 才能下結論。

AIOrchestrations 概觀 – 機制 2



①路由（機制 1） ②行為契約（機制 2） ③安全邊界（機制 3） ④自我改善閉環（機制 4）

行為契約：五條規則

1 先講完成條件

動手前先定義可觀察、可測試的「完成」，不是「感覺可以了」

2 驗證要誠實

沒跑過的檢查不能宣稱已驗證；指令成功≠問題真的解決

3 標記未驗證

查不到的事實明確標成 Unverified，不當成已知

4 失敗兩次就換方法

同一做法失敗兩次，停下來重新界定問題

5 依風險決定驗證力度

小事輕量驗證、高風險才走完整流程；失敗一次就升級

🤔 猜猜看

案例測試：AI 說測了 4 種情境，全部「被回收」

你會怎麼回覆 AI？

Scenario	Collected
S1 XAML Style animation	yes
S2 C# RunAsync + Token	yes
S3 C# RunAsync, no Token	yes
S4 XAML, IsVisible=false	yes

A 沒有洩漏，可以結案

B 先確認「動畫真的有在跑」

C 再多跑幾次再說

👋 請舉手

全數回收，不代表動畫已驗證

Scenario	Ran	Collected
S1 XAML Style animation	NO	yes
S2 C# RunAsync + CancellationToken	NO	yes
S3 C# RunAsync, no CancellationToken	NO	yes
S4 XAML, IsVisible=false	NO	yes

Ran = 動畫真的有跑嗎？

只看 Collected 會得到

「全部都被回收，沒有洩漏」

但 Ran 欄顯示：測試中沒有觀察到動畫執行。

原因：Headless 進行測試時，無 UI 啟動，因此無法驗證 UI 動畫。

① 完成條件：動畫必須實際執行
② 驗證要誠實：Ran=NO，不可宣稱已驗證沒有洩漏
③ 標記未驗證：UI 是否啟動執行動畫，Headless 無法判定（Unverified）

🤔 猜猜看

同一招，已經失敗兩次了。第三次，你（或 AI）會.....

A

再試一次，也許只是運氣

B

加個特例，先讓它過

C

停下來，寫出失敗原因 X（證據）

👋 請舉手

同一招失敗兩次，就停止

AI 要重試前，必須能填完這句話：

> **失敗原因是 X（證據：……），**
> **修改 Y 會修正 X，因為 Z。**

- 填不出「證據」→ 先蒐證，不要再試一次
- 兩次嘗試失敗的方式不同 → 前提可能是錯的，重新推導
- 為了保護目前做法而一直加特例 → 該重新設計的訊號

依風險決定驗證的等級，再用檢查表收尾

每個任務開始前，請 AI 先評估影響、可逆性與出錯後果，再標定要執行的風險等級；高風險先請人確認。

等級	什麼時候
LIGHT	≤2 個檔案、小而可逆、有機械式檢查
STANDARD	預設：完整契約
FULL	共用規則、不可逆、或對外可見的動作

檢查表（節錄）

- ☐ 輸出真的存在
- ☐ 每條準則有證據
- ☐ 原本失敗的檢查重跑通過
- ☐ 未驗證項目已列出
- ☐ 敏感暫存檔已清理

LIGHT 失敗一次，就升級為 STANDARD

任務：查測試機台近 48 小時有無運作錯誤

情境推演：AI 先初評風險；高風險操作執行前必須由人確認。

⑤ 開始前 | AI 先評估風險

範圍：單一測試機台 × 近 48 小時

操作：只讀查詢，不改機台設定

AI 先評估漏報影響，再決定查證力度

做法：交叉核對 log / 告警，註明來源與時間

若要重啟或改設定，先請人確認

④ 執行中 | 同一查法失敗兩次

停手：不再重試同一查詢方式

改查其他資料來源，或補足存取條件

③ 查不到 → 標記 Unverified
不能把「查不到」回報成「沒有錯誤」

本例對應：③查不到標記 Unverified　④同一方法失敗兩次就停手　⑤依風險決定驗證力度

AIOrchestrations 概觀 – 機制 3



①路由（機制 1） ②行為契約（機制 2） ③安全邊界（機制 3） ④自我改善閉環（機制 4）

Skill 先宣告邊界，再談能力

- 允許的動作
- 會有哪些副作用
- 是否需要人工核准
- 明確不可碰的資料

```
## Safety boundary

- Scope: [target / environment]

- Allowed: [actions / read-only]

- Inputs: [required; user-supplied]

- Side effects: [none / exact impact]

- Needs approval: [action / approver] (write actions: yes/no)

- Never touch: [credentials / protected data]

- Validation: [observable check; unseen = Unverified]
```

規則訂在 Repo 中，但處理過程中所產出資料不准進

AIOrchestrations repo（可審查）

workflows · agents · skills
prompts · knowledge · scripts
docs · schemas

不進 repo（runtime）

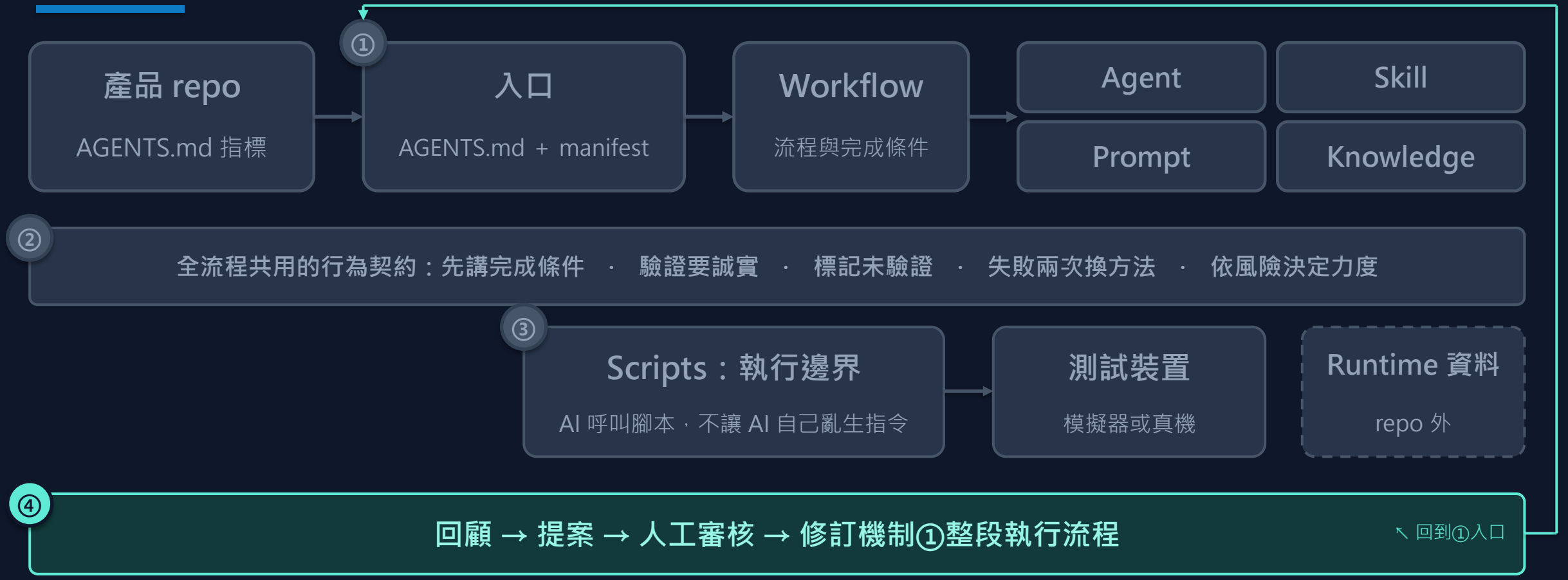
artifacts · downloads · logs
workspaces
config/user.local.json

- 不在對話裡貼密碼、Token、金鑰
- 只匯出短效憑證，用完即刪；可提交**脫敏**後的範例讓 AI 之後可進行 CoT (思維鏈) 處理

越靠近底層，護欄會越具體

- 腳本是 AI 的執行邊界：AI 呼叫腳本，而不是讓 AI 自己拼指令執行
- 參數明確化、輸入做引號處理，拒絕夾帶其他指令的原始參數
- 高風險動作：先核准 → 執行 → 回讀驗證
- 現場出過的問題，要回頭改寫 Skill 的安全設計

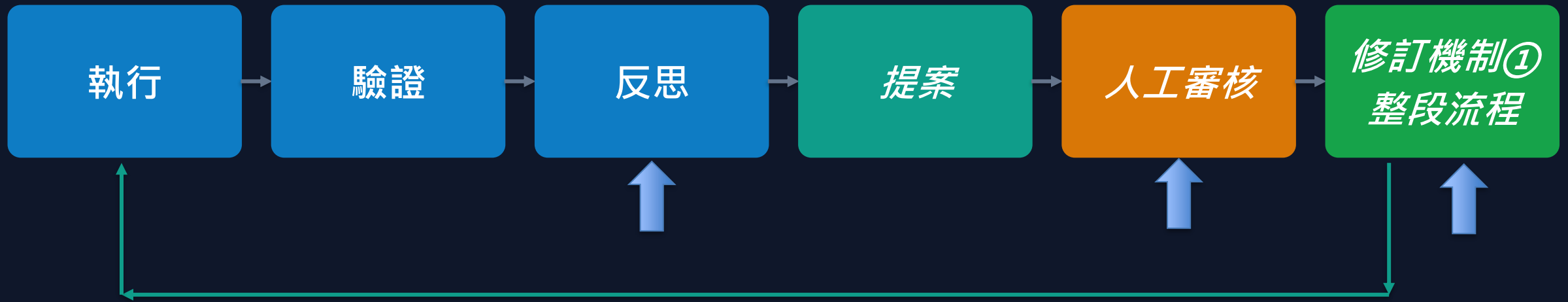
AIOrchestrations 概觀 – 機制 4



①路由（機制 1） ②行為契約（機制 2） ③安全邊界（機制 3） ④自我改善閉環（機制 4）

會自我改善的 Loop

觸發時機：有值得沉澱的任務完成時，再啟動*回顧流程*



核准修訂後，後續任務就能依更新後的機制①載入與執行

AI 只能協助「提案」，由人把關 MR / PR 與確認修訂

提案寫出來，不等於規則

① 人提出整理指示

Session 中明確提出

幫我整理成可進入
AIOrchestrations 的處理

② 人審核提案

AI 草擬，供人判斷

逐項判斷是否採納
未核准，不改共用檔

③ 核准後才更新

依核准結果走更新流程

修改共用檔案
審查通過才成為規則

AI 負責整理與草擬；人負責判斷與決定

最小導入藍圖

Day 1

讓 AI 找對規則

AGENTS.md 入口檔
一頁行為契約

Week 1

把重複任務流程化

Workflow + Skill
含安全邊界

Month 1

讓流程持續可檢查

回顧提案流程
一致性檢查（CI）

全部都是 Git、文件、腳本，不需要新平台。

三件事，帶回去

① 把 Prompt 變成資產

② 完成，要有可觀察的證據

③ 演進，由人把關

延伸閱讀：iT 邦幫忙 2026 鐵人賽《當 AI 加入團隊》30 篇
掃描 QR code 延伸閱讀

https://ithelp.ithome.com.tw/users/20126714/ironman/9668

Thank you !

Q & A

現成技術或工具，能再多做哪些？

Azure AI Search

搜尋 / 索引

關鍵字 + 向量混合檢索；補搜尋彈性
仍需管理索引更新與權限

Google Agent Search

原 Vertex AI Search | 企業搜尋 / RAG

代管企業資料搜尋與 RAG grounding
仍要治理資料來源、權限與成本

Atlassian Rovo

企業知識搜尋 / Agent

跨 Atlassian 與已連接的資料來源
能力受連接器與平台生態影響

CI Pipeline

GitHub Actions / GitLab CI/CD

PR / MR 自動檢 schema、路徑、連結
攔結構錯誤；語意仍需人工審查

服務能接手搜尋、索引與機械檢查；規則取捨與核准仍由人把關。

官方文件：Azure AI Search · Google Agent Search · Atlassian Rovo · GitHub Actions · GitLab CI/CD